Freestanding Feature-Test Macros and Implementation-Defined Extensions

Document number: P2198R6
Date: 2022-12-06
Reply-to: Ben Craig <ben dot craig at gmail dot com>
Audience: Library Working Group

Change history

R6

- Rebase to N4917
- Marking the following new feature test macros as freestanding: `__cpp_lib_forward_like`, `__cpp_lib_modules`, `__cpp_lib_move_iterator_concept`, `__cpp_lib_ranges_as_const`, `__cpp_lib_ranges_as_rvalue`, `__cpp_lib_ranges_cartesian_product`, `__cpp_lib_ranges_repeat`, `__cpp_lib_ranges_stride`, and `__cpp_lib_start_lifetime_as`.

R5

- Preemptively deal with in-flight LWG papers
- Marking the following new feature test macros as freestanding: `__cpp_lib_bind_back`, `__cpp_lib_ranges_chunk`, `__cpp_lib_ranges_chunk_by`, `__cpp_lib_ranges_join_with`, `__cpp_lib_ranges_slide`, `__cpp_lib_ranges_to_container`, `__cpp_lib_reference_from_temporary`, and `__cpp_lib_unreachable`.

R4

- Wording updates and LEWG feedback
- No longer bumping all the non-freestanding macros, and instead adding a new macro to indicate the old macros aren't lying
- Changing Compiler Explorer example to longer mention GCC's freestanding implementation, as that requires a custom libstdc++ build

R3

- Rebase to N4901
- Only bumping the versions of C++20 (and older) feature test macros
- Marking the following new feature test macros as freestanding: `__cpp_lib_byteswap`, `__cpp_lib_constexpr_typeinfo`, `__cpp_lib_invoke_r`, `__cpp_lib_is_scoped_enum`, `__cpp_lib_ranges_zip`, and `__cpp_lib_to_underlying`

R2

- Corrected policy recommendations to match examples and wording for `::operator new` example.
- Updated wording tying feature test macro to `::operator new` behavior.

R1

- Indicating which working draft the wording was based on.
- Indicating the dependence on P1642 wording.
- Added macro for detecting if `::operator new` has a library provided definition (related to P2013R3).

R0

- Branching from P1641R3 and P1642R3.
- Freestanding feature-test macros now required to be present in hosted.
- Added examples and use cases.
- Elaborating on how an implementation can extend the bare minimum of freestanding.

Introduction

Users of freestanding implementations would like to be able to detect which library facilities are available for use. With the C++20 feature-test macros, all the feature-test macros are required to be defined in both freestanding and hosted implementations, even when the associated hosted facilities are not available. This paper will make the presence of feature-test macros for hosted facilities implementation-defined.

This paper also clarifies that freestanding implementations need not stop at the bare minimum support for freestanding. Freestanding implementations may provide additional facilities from the standard library, so long as those facilities meet the associated hosted requirements.

Motivation and Design

Beyond the bare minimum

This paper grants implementations the freedom to include more than just the facilities that are marked `//freestanding`. There will be platforms where, for example, floating point is both available and frequently used (e.g. GPU environments). On those platforms, it is desirable for implementations to provide many of the facilities in the `<cmath>` and `<complex>` headers. We must be careful here though, as we don't want to allow divergence of implementation, and we want to be able to add facilities to the required subset of freestanding. To permit both vendor extension and future freestanding growth, this paper will require that additionally included facilities must meet the same requirements as for a hosted implementation, with a few exceptions.

Freestanding implementations may include additional macros and namespace scoped entities, beyond what is required in a minimal freestanding implementation. A freestanding implementation does not need to provide the entirety of a header. If a freestanding implementation supplies an additional class, the entirety of the class must be present and meet all the hosted requirements. This prevents some useful vendor extensions, notably `at()`-less `array`, `value()`-less `optional`, and similarly partial `string_view` and `bitset`. This does not prevent us from standardizing a partial class in the future, though it does prevent us from getting usage experience in a conforming implementation.

Freestanding implementations may make the use of any non-freestanding, namespace scoped function or function template ill-formed (e.g. generally by marking it `=delete`). The intent is to allow the freestanding implementation to control overload resolution, so that a library that works with a freestanding implementation will have the same semantics in a hosted implementation.

Users will be able to rely on the freestanding portions of the standard library. The freestanding portions have portable semantics, and are required to be present. Users may then decide to rely on implementation-defined additions. WG21 will be able to add hosted entities to the freestanding subset without fear of breaking implementation-defined additions, because the implementation-defined additions needed to meet the hosted requirements.

Detecting once-required features

P2013R3 makes the default allocating `::operator news` optional. Users may want to detect this in their code so that they can fall-back to a fixed capacity container rather than a dynamically sized container. Some users may also want to (ab)use such a feature-test macro so that they can provide an `::operator new` implementation when one is not provided by the standard library.

This paper recommends a macro to detect the presence of `::operator new` definitions: `__cpp_lib_freestanding_operator_new`. Unlike other feature-test macros, this macro will be required to be set to `0` on freestanding implementations that do not provide useful `::operator new` definitions. This will allow users to detect whether the library is an old standard library or a new standard library as well.

Unlike other feature-test macros, this macro will need normative wording to tie the feature-test macro to the feature that it is describing.

Meta feature-test macro

C++20 requires that all implementations (freestanding and hosted) define all of the library feature-test macros in the `<version>` header, even the feature-test macros that correspond to facilities not required to be present in freestanding implementations. This means that those feature-test macros provide misleading results on freestanding implementations. This isn't just a theoretical problem. Existing implementations are already deploying `<version>` headers that report support for `std::filesystem` facilities, `std::chrono` facilities, and many others, even though those feature-test macros indicate support for features that require the support of an operating system.

In order for users of freestanding implementations to be able to detect extensions of freestanding, the users need a way of distinguishing the C++20 macro requirements from an accurate expression of extension. This paper will provide a new macro,

`__cpp_lib_freestanding_feature_test_macros`, so that users can distinguish between these cases. When `__cpp_lib_freestanding_feature_test_macros` is present, users will know that the C++20 feature-test macros aren't lying.

Freestanding feature-test macros

The following macros are now required to be present in freestanding implementations. The other library feature test macros in the `<version>` header are not required to be present on freestanding implementations. The corresponding facilities were either required to be present in C++20, or are added in P1642. The green feature-test macros are the ones added since the paper was forwarded from LEWG.

- `__cpp_lib_addressof_constexpr`
- `__cpp_lib_allocator_traits_is_always_equal`
- `__cpp_lib_apply`
- `__cpp_lib_as_const`
- `__cpp_lib_assume_aligned`
- `__cpp_lib_atomic_flag_test`
- `__cpp_lib_atomic_float`
- `__cpp_lib_atomic_is_always_lock_free`
- `__cpp_lib_atomic_ref`
- `__cpp_lib_atomic_value_initialization`
- `__cpp_lib_atomic_wait`
- **`__cpp_lib_bind_back`**
- `__cpp_lib_bind_front`
- `__cpp_lib_bit_cast`
- `__cpp_lib_bitops`
- `__cpp_lib_bool_constant`
- `__cpp_lib_bounded_array_traits`
- `__cpp_lib_byte`
- **`__cpp_lib_byteswap`**
- `__cpp_lib_char8_t`
- `__cpp_lib_concepts`
- `__cpp_lib_constexpr_functional`
- `__cpp_lib_constexpr_iterator`
- `__cpp_lib_constexpr_memory`
- `__cpp_lib_constexpr_tuple`
- **`__cpp_lib_constexpr_typeinfo`**
- `__cpp_lib_constexpr_utility`
- `__cpp_lib_destroying_delete`
- `__cpp_lib_endian`
- `__cpp_lib_exchange_function`
- **`__cpp_lib_forward_like`**
- `__cpp_lib_hardware_interference_size`
- `__cpp_lib_has_unique_object_representations`
- `__cpp_lib_int_pow2`
- `__cpp_lib_integer_sequence`
- `__cpp_lib_integral_constant_callable`
- `__cpp_lib_invoke`
- **`__cpp_lib_invoke_r`**
- `__cpp_lib_is_aggregate`
- `__cpp_lib_is_constant_evaluated`
- `__cpp_lib_is_final`
- `__cpp_lib_is_invocable`
- `__cpp_lib_is_layout_compatible`
- `__cpp_lib_is_nothrow_convertible`
- `__cpp_lib_is_null_pointer`
- `__cpp_lib_is_pointer_interconvertible`
- **`__cpp_lib_is_scoped_enum`**
- `__cpp_lib_is_swappable`
- `__cpp_lib_launder`
- `__cpp_lib_logical_traits`
- `__cpp_lib_make_from_tuple`
- `__cpp_lib_make_reverse_iterator`
- **`__cpp_lib_modules`**

- **__cpp_lib_move_iterator_concept**
- __cpp_lib_nonmember_container_access
- __cpp_lib_not_fn
- __cpp_lib_null_iterators
- **__cpp_lib_ranges_as_const**
- **__cpp_lib_ranges_as_rvalue**
- **__cpp_lib_ranges_cartesian_product**
- **__cpp_lib_ranges_chunk**
- **__cpp_lib_ranges_chunk_by**
- **__cpp_lib_ranges_join_with**
- **__cpp_lib_ranges_repeat**
- **__cpp_lib_ranges_slide**
- **__cpp_lib_ranges_stride**
- **__cpp_lib_ranges_to_container**
- **__cpp_lib_ranges_zip**
- **__cpp_lib_reference_from_temporary**
- __cpp_lib_remove_cvref
- __cpp_lib_result_of_sfinae
- __cpp_lib_source_location
- __cpp_lib_ssize
- **__cpp_lib_start_lifetime_as**
- __cpp_lib_three_way_comparison
- __cpp_lib_to_address
- **__cpp_lib_to_underlying**
- __cpp_lib_transformation_trait_aliases
- __cpp_lib_transparent_operators
- __cpp_lib_tuple_element_t
- __cpp_lib_tuples_by_type
- __cpp_lib_type_identity
- __cpp_lib_type_trait_variable_templates
- __cpp_lib_uncaught_exceptions
- **__cpp_lib_unreachable**
- __cpp_lib_unwrap_ref
- __cpp_lib_void_t

Post LEWG additions

These papers with freestanding facilities were approved after this paper left LEWG. This paper was included after a September 2021 reflector discussion, and it received five votes in favor, with no opposition (the author of this paper did not vote).

- P0627R6: Function to mark unreachable code

These papers were included after a November 2021 reflector discussion, and they received five votes in favor, with no opposition (the author of this paper did not vote).

- P2136R3: invoke_r
- P2321R2: zip
- P1682R3: std::to_underlying

These papers were included after an April 2022 reflector discussion, and they received six votes in favor, with no opposition (the author of this paper did not vote).

- P1206R7 ranges::to
- P1899 views::stride
- P2165 Compatibility Between tuple, pair, And tuple-Like Objects
- P2278 cbegin should always return a constant iterator
- P2374 views::cartesian_product
- P2387R3 Pipe support for user-defined range adaptors
- P2441R2 views::join_with
- P2442R1 Windowing range adaptors: views::chunk and views::slide
- P2443R1 views::chunk_by
- P2445 forward_like
- P2446 views::all_move
- P2474 views::repeat

- P2494 Relaxing Range Adaptors To Allow For Move Only Types

New feature-test macros

Users of freestanding implementations will want to know whether including a formerly hosted-only header will work or not. Users of freestanding implementations will also what to know if all the facilities that are required to be in freestanding have been made available yet. This is a concern for highly portable libraries, and for users that need to support old and new compilers.

These feature-test macros are provided at a per-header granularity. This enables implementations to advertise new capabilities more easily than a single feature-test macro for the entirety of this paper. This also follows the precedent set by the `__cpp_lib_constexpr_*` macros.

If new, pre-C++20 functionality is added to the freestanding subset of C++, then the respective feature-test macro for the header should be bumped. If the functionality is new in C++23 or later, then alternative approaches should be taken. These alternative approaches are discussed in the examples section.

Name	Header
`__cpp_lib_freestanding_feature_test_macros`	only `<version>`
`__cpp_lib_freestanding_utility`	`<utility>`
`__cpp_lib_freestanding_tuple`	`<tuple>`
`__cpp_lib_freestanding_ratio`	`<ratio>`
`__cpp_lib_freestanding_memory`	`<memory>`
`__cpp_lib_freestanding_functional`	`<functional>`
`__cpp_lib_freestanding_iterator`	`<iterator>`
`__cpp_lib_freestanding_ranges`	`<ranges>`

Partial macro coverage

The following, existing feature-test macros cover some features that I am making freestanding, and some features that I am not requiring to be freestanding. These feature-test macros won't be required in freestanding, as they could cause substantial confusion when the hosted parts of those features aren't available. The per-header `__cpp_lib_freestanding_*` macros should provide a suitable replacement in freestanding environments.

- `__cpp_lib_boyer_moore_searcher`: `default_searcher` is in, other searchers require the heap.
- `__cpp_lib_constexpr_dynamic_alloc`: `constexpr`ification of various memory algorithms is in, anything dealing with `std::allocator` is out
- `__cpp_lib_ranges`: stream iterators are out
- `__cpp_lib_raw_memory_algorithms`: ExecutionPolicy overloads are out

Feature-test macro policy recommendations

This paper patches up many of the historical problems with freestanding and feature-test macros. The following are the guidelines I recommend to keep feature-test macros useful for freestanding in the future.

- If a paper contains some facilities that are freestanding, and some that are hosted, then the paper needs at least two feature test macros. `__cpp_lib_foo` will cover the full paper in the hosted case, and `__cpp_lib_freestanding_foo` will cover the freestanding portions.
- If a paper adds facilities to the hosted implementation and has a feature-test macro, and a later paper adds the entirety of the paper to freestanding, then bump the version of the original feature-test macro.
- If a paper adds facilities to the hosted implementation and has a feature-test macro, and a later paper adds a portion of the paper to freestanding, then add a new freestanding macro for the feature, of the form `__cpp_lib_freestanding_foo`.
- If a paper changes a freestanding facility from required to optional, then add a "presence" feature-test macro of the form `__cpp_lib_freestanding_foo`. Define the macro to 0 when the feature is not present, and add normative wording tying the feature to the feature test macro.

Examples

The following feature detection examples should ...

- ... work on freestanding, hosted, and with implementation-defined freestanding extensions.
- ... work with old standards and new standards.

- ... provide a conservative "false" answer when working with an old standard.
- ... not cause warnings for the use of an undefined macro, like with `-Wundef`.

The examples will take advantage of a helper macro as well:

```
#if (__STDC_HOSTED__ || \
     (defined(__cpp_lib_freestanding_feature_test_macros) && \
      __cpp_lib_freestanding_feature_test_macros >= 202112))
#define TRUTHFUL_MACROS 1
#else
#define TRUTHFUL_MACROS 0
#endif
```

Detect C++20 (or older) feature that is not required in freestanding

Is `filesystem::copy` *available?*

```
#if TRUTHFUL_MACROS && defined(__cpp_lib_filesystem) && __cpp_lib_filesystem >= 201703L
// hosted and freestanding extension success path
#else
// fallback path
#endif
```

Detect C++20 (or older) feature that is now entirely required in freestanding

Is `ssize` *available?*

```
#if TRUTHFUL_MACROS && defined(__cpp_lib_ssize) && __cpp_lib_ssize >= 201902L
// hosted and freestanding success path
#else
// fallback path
#endif
```

Detect C++20 (or older) feature that is now partially required in freestanding

Is the non-parallel version of `uninitialized_default_construct` *available?*

```
#if TRUTHFUL_MACROS && defined(__cpp_lib_raw_memory_algorithms) && __cpp_lib_raw_memory_algorithms >= 201606L
// hosted and freestanding extension success path
#elif defined(__cpp_lib_freestanding_memory) && __cpp_lib_freestanding_memory >= 202007L
// freestanding success path
#else
// fallback path
#endif
```

Detect pre-feature-test-macro feature that is required in freestanding

Is `tuple` *available?*

```
#if defined(__cpp_lib_freestanding_tuple) && __cpp_lib_freestanding_tuple >= 202007L
// freestanding and future hosted success path
#elif __STDC_HOSTED__
// flakey hosted success path.  Assume tuple works here
#else
// fallback path
#endif
```

Detect pre-feature-test-macro feature that is not required in freestanding

Is `thread` *available?*

```
// No good answers here.  Currently calling this out of scope.
```

Detect C++23 feature that is entirely freestanding in the initial paper

```
#if defined(__cpp_lib_always_freestanding_feature) && __cpp_lib_always_freestanding_feature >= 202202L
// hosted and freestanding success path
#else
```

```
// fallback path
#endif
```

Detect C++23 feature that is hosted in the initial paper, but made entirely freestanding later (possibly in another version of C++)

For this category of features, the recommendation will be to bump the integer constant in the version test macro.

```
#if defined(__cpp_lib_eventually_freestanding_feature)
    #if __cpp_lib_eventually_freestanding_feature >= 202702L
    // freestanding success path.  Will also trigger for new hosted toolchains
    #elif __STDC_HOSTED__ && __cpp_lib_eventually_freestanding_feature >= 202102L
    // Interim hosted success path
    #else
    // Interim freestanding fallback path
    #endif
#else
// fallback path
#endif

// Alternative that will work 99% of the time
#if defined(__cpp_lib_eventually_freestanding_feature)
// freestanding and hosted success, probably
#else
// fallback path
#endif
```

Detect freestanding part of C++23 feature that is partially freestanding in the initial paper

For this category of features, the recommendation will be to have two macros, one for the freestanding portion, and one for the entire paper. The macros should follow the convention `__cpp_lib_foo` for the full paper, and `__cpp_lib_freestanding_foo` for the freestanding portion.
Is the freestanding portion of feature foo available?

```
#if defined(__cpp_lib_freestanding_foo) && __cpp_lib_freestanding_foo >= 202202L
// hosted and freestanding success path
#else
// fallback path
#endif
```

Detect freestanding part of C++23 feature that is made freestanding after the initial paper (possibly in a different C++ release)

For this category of features, the recommendation will be to add a new freestanding macro for the feature, following the convention of `__cpp_lib_freestanding_bar` for the freestanding portion.
Is the freestanding portion of feature bar available?

```
#if defined(__cpp_lib_bar) && __cpp_lib_bar >= 202202L
// Old hosted toolchain path and freestanding extension path
#elif defined(__cpp_lib_freestanding_bar) && __cpp_lib_freestanding_bar >= 202702L
// freestanding success path.  Will also trigger for new hosted toolchains
#else
// fallback path
#endif
```

Detect absence of a previously required freestanding feature

Is `::operator new`*'s definition in the standard library?*

```
#if defined(__cpp_lib_freestanding_operator_new)
    #if __cpp_lib_freestanding_operator_new >= 202009L
    // ::operator new is available
    #else
    // No ::operator new available
    static_assert(__cpp_lib_freestanding_operator_new == 0)
    #endif
#elif (/*vendor specific test*/)
// old, non-conforming freestanding "success" path.  No ::operator new available.
```

```
#else
// old hosted and heap-capable freestanding code path.  ::operator new is available.
#endif
```

Rejected alternatives

Bump existing macros instead of provide a new macro

Users need a bit of information to know whether the C++20 macros are lying or not. Prior revisions of this paper provided that bit by bumping the version number of the C++20 macros. This works, but it expands the scope of this change. It also interacts poorly when there are multiple changes to the same macro.

Instead, we provide the bit of information with the new `__cpp_lib_freestanding_feature_test_macros` macro.

Allow partial classes in conforming vendor extensions to freestanding

Allowing partial classes gives implementers a great deal of freedom to experiment and to provide extensions. It also permits abuse by the implementers. On the positive side, an implementation could provide `std::array` without `at`. This helps replace C facilities with better facilities from C++. On the negative side, an implementation could `=delete` the copy constructor. Removing the wrong functions would also mean that a class may not satisfy the same concepts in hosted vs. freestanding.

Update old feature-test macros to indicate freestanding status

Rather than have a `__cpp_lib_freestanding_ratio` (for example), we could instead update `__cpp_lib_chrono`. Users could test the value of `__cpp_lib_chrono` to determine if it is freestanding safe or not.

There are at least two big flaws with this approach.

First, not all facilities (particularly older ones) have existing feature-test macros to bump (e.g. `unique_ptr`). Users would still like to be able to detect the availability of the features in freestanding.

Second, implementers don't always implement features in the same order that they are added to the working draft. If a new feature were added to the `<chrono>` header that necessitated a feature-test macro bump, and an implementer addressed that feature before making the `<ratio>` header freestanding safe, then the implementer would either need to stick with the pre-freestanding feature-test macro version, or provide a misleading feature-test macro version.

Per-freestanding paper feature-test macro granularity

Rather than introduce a feature-test macro per header in this paper, I could instead introduce one feature-test macro... perhaps `__cpp_lib_freestanding`. Each paper that adds old facilities to freestanding could then introduce it's own macro, or bump the old one.

This approach can work, but it restricts the order in which implementers can meaningfully implement freestanding features. All of the old paper needs to be done before any of the old paper's progress can be advertised. All of the old paper needs to be done before advertising any newer papers.

Per-facility feature-test macro granularity

This paper could choose to add a feature-test macro for `__cpp_lib_freestanding_unique_ptr`, `__cpp_lib_freestanding_pair`, and `__cpp_lib_freestanding_iterator_categories`. There would be some value to users in that they could express exactly what it is they need, and see if that very specific facility is available.

However, this approach is an implementation hassle, and prone to endless wg21 debates on how to partition and name the facilities. Grouping facilities by header provides a natural partitioning and naming scheme.

Rely on `__has_include`

Some have suggested using `__has_include` to detect whether the `<tuple>` header (for example) is usable on a particular freestanding implementation. This doesn't work in practice for multiple reasons.

The first example is the `<iterator>` header. In older versions of Visual Studio, a user could attempt to `#include <iterator>`. The streaming iterators in the header result in compiler errors. We need feature-test macros to indicate whether including the header is well-formed.

The second example is headers that do standard versions checks inside. The libstdc++ implementation of the `<ranges>` header is mostly empty if the language version is less than C++20. `__has_include` will still report the header as present though.

Make `__cpp_lib_freestanding_*` macros freestanding only

This complicates the client feature-test code for features that were marked freestanding in their initial papers. It doesn't make any of the client feature-test code any simpler. It also violates the principle that freestanding should be a subset of hosted.

Define `__cpp_lib_freestanding_*` macros to 0 in hosted

Users of the freestanding macros would still need to account for the case where the macro isn't present. Defining the macro to zero doesn't provide any information beyond what `__STDC_HOSTED__` provides.

Detect once-required features with a feature-test "absence" macro

We could define a feature-test macro in the negative, for example `__cpp_lib_freestanding_no_operator_new`. This would be different from every other feature-test macro in the standard and SD-6 (SD-6's `__cpp_rtti` and `__cpp_exceptions` are defined in the positive). Hosted implementations would never define this macro. Freestanding implementations may or may not define the macro. When the user detects this "absence" macro, they could take action confidently. When a user fails to detect a "presence" macro, the user would need to carefully consider whether it is because of an old standard library or a missing `::operator new` definition.

This approach would still be useful "soon", but it would leave the ambiguity of the pre-adoption and post-adoption states.

Detect once-required features with a traditional feature-test macro

We could have a "presence" feature-test macro that is undefined when not set, as opposed to set to `0`, as is proposed. Unfortunately, it would take a long time for this macro to be useful in practice. Most implementations today have an `::operator new` definition available, but don't have this macro defined. That means that this macro would only be useful in the distant future, where the absence of this macro is more likely to indicate a system without a heap than a system with an old standard library. Setting the macro to `0` when the feature is not present is critical to the usability.

Wording

The following wording is relative to N4917, and assumes that P2013 have been applied.

Additions to [compliance]

Please append the following paragraphs to [compliance].

5 The *hosted library facilities* are the set of facilities described in this document that are required for hosted implementations, but not required for freestanding implementations. A freestanding implementation provides a (possibly empty) implementation-defined subset of the hosted library facilities. Unless otherwise specified, the requirements on each declaration, entity, *typedef-name*, and macro provided in this way shall be the same as the corresponding requirements for a hosted implementation, except that not all of the members of the namespaces are required to be present.

6 A freestanding implementation provides deleted definitions for a (possibly empty) implementation-defined subset of the namespace scoped functions and function templates from the hosted library facilities.
[*Note:* An implementation may provide a deleted definition so that overload resolution does not silently change when migrating a library from a freestanding implementation to a hosted implementation. -*end note*]

Change in [version.syn]

Add "`freestanding ,`" to the beginning of each comment in the following macro definitions in [version.syn]:

- `__cpp_lib_addressof_constexpr`
- `__cpp_lib_allocator_traits_is_always_equal`
- `__cpp_lib_apply`
- `__cpp_lib_as_const`
- `__cpp_lib_assume_aligned`

- __cpp_lib_atomic_flag_test
- __cpp_lib_atomic_float
- __cpp_lib_atomic_is_always_lock_free
- __cpp_lib_atomic_ref
- __cpp_lib_atomic_value_initialization
- __cpp_lib_atomic_wait
- __cpp_lib_bind_back
- __cpp_lib_bind_front
- __cpp_lib_bit_cast
- __cpp_lib_bitops
- __cpp_lib_bool_constant
- __cpp_lib_bounded_array_traits
- __cpp_lib_byte
- __cpp_lib_byteswap
- __cpp_lib_char8_t
- __cpp_lib_concepts
- __cpp_lib_constexpr_functional
- __cpp_lib_constexpr_iterator
- __cpp_lib_constexpr_memory
- __cpp_lib_constexpr_tuple
- __cpp_lib_constexpr_typeinfo
- __cpp_lib_constexpr_utility
- __cpp_lib_destroying_delete
- __cpp_lib_endian
- __cpp_lib_exchange_function
- __cpp_lib_forward_like
- __cpp_lib_hardware_interference_size
- __cpp_lib_has_unique_object_representations
- __cpp_lib_int_pow2
- __cpp_lib_integer_sequence
- __cpp_lib_integral_constant_callable
- __cpp_lib_invoke
- __cpp_lib_invoke_r
- __cpp_lib_is_aggregate
- __cpp_lib_is_constant_evaluated
- __cpp_lib_is_final
- __cpp_lib_is_invocable
- __cpp_lib_is_layout_compatible
- __cpp_lib_is_nothrow_convertible
- __cpp_lib_is_null_pointer
- __cpp_lib_is_pointer_interconvertible
- __cpp_lib_is_scoped_enum
- __cpp_lib_is_swappable
- __cpp_lib_launder
- __cpp_lib_logical_traits
- __cpp_lib_make_from_tuple
- __cpp_lib_make_reverse_iterator
- __cpp_lib_move_iterator_concept
- __cpp_lib_nonmember_container_access
- __cpp_lib_not_fn
- __cpp_lib_null_iterators
- __cpp_lib_ranges_as_const
- __cpp_lib_ranges_as_rvalue
- __cpp_lib_ranges_cartesian_product
- __cpp_lib_ranges_chunk
- __cpp_lib_ranges_chunk_by
- __cpp_lib_ranges_join_with
- __cpp_lib_ranges_repeat
- __cpp_lib_ranges_slide
- __cpp_lib_ranges_stride
- __cpp_lib_ranges_to_container
- __cpp_lib_ranges_zip
- __cpp_lib_reference_from_temporary
- __cpp_lib_remove_cvref

- `__cpp_lib_result_of_sfinae`
- `__cpp_lib_source_location`
- `__cpp_lib_ssize`
- `__cpp_lib_start_lifetime_as`
- `__cpp_lib_three_way_comparison`
- `__cpp_lib_to_address`
- `__cpp_lib_to_underlying`
- `__cpp_lib_transformation_trait_aliases`
- `__cpp_lib_transparent_operators`
- `__cpp_lib_tuple_element_t`
- `__cpp_lib_tuples_by_type`
- `__cpp_lib_type_identity`
- `__cpp_lib_type_trait_variable_templates`
- `__cpp_lib_uncaught_exceptions`
- `__cpp_lib_unreachable`
- `__cpp_lib_unwrap_ref`
- `__cpp_lib_void_t`

The changes should look similar to this:

```
2   #define __cpp_lib_addressof_constexpr              201603L // freestanding, also in <memory>
```

Add a "`// freestanding`" comment after the following macro definition in [version.syn]:

- `__cpp_lib_modules`

Please add the following macros to [version.syn]/2. Use the current year-based constant instead of "*new-val*".

```
2   #define __cpp_lib_freestanding_feature_test_macros  new-val // freestanding
    #define __cpp_lib_freestanding_functional           new-val // freestanding, also in <functional>
    #define __cpp_lib_freestanding_iterator             new-val // freestanding, also in <iterator>
    #define __cpp_lib_freestanding_memory               new-val // freestanding, also in <memory>
    #define __cpp_lib_freestanding_operator_new       see below // freestanding, also in <new>
    #define __cpp_lib_freestanding_ranges               new-val // freestanding, also in <ranges>
    #define __cpp_lib_freestanding_ratio                new-val // freestanding, also in <ratio>
    #define __cpp_lib_freestanding_tuple                new-val // freestanding, also in <tuple>
    #define __cpp_lib_freestanding_utility              new-val // freestanding, also in <utility>
```

Please append the following paragraphs to [version.syn].

3 The macro `__cpp_lib_freestanding_operator_new` is defined to the integer literal *`new-val`* if all of the library provided replaceable global allocation functions meet the requirements of a hosted implementation, and to the integer literal `0` otherwise. ([new.delete]).

4 *Recommended practice:* Freestanding implementations should only define a macro from `<version>` if the implementation provides the corresponding facility in its entirety.